

Administrador de Música

Escuela Superior de Física y Matemáticas

9 de octubre de 2026

Índice de Contenidos

SQL

- Modelado de Base de Datos

- Entidades/Tablas principales

- Relaciones

- Modelo de relación

- Ejemplo

Python

- Archivos

- Recursos: Librería `mysql.connector`

Python

- Archivos del Proyecto

- Conexión a la Base de Datos

- Funciones de la Aplicación

- Controlador Principal (`main.py`)

¿Cuál es el objetivo?

El objetivo de esta estructura es relacionar las entidades para normalizar la base de datos y optimizar el almacenamiento.

La tabla **álbumes** funciona como la entidad principal del catálogo. Para evitar duplicidad de información, los registros no se insertan en un único bloque; primero se registran las entidades independientes y posteriormente se vinculan a través de tablas relacionales intermedias.

Gracias a esto, un artista puede tener varios álbumes, un álbum puede reunir varias canciones y una canción puede pertenecer a más de un lanzamiento, evitando en todo momento la redundancia de datos.

Entidades

usuarios

Campo	Tipo
id_usuario	INT (PK)
username	VARCHAR(50)
password_hash	VARCHAR(255)
created_at	TIMESTAMP

artistas

Campo	Tipo
id_artista	INT (PK)
nombre	VARCHAR(150)

albumes

Campo	Tipo
id_album	INT (PK)
titulo	VARCHAR(150)
descripcion	TEXT
anio_lanzamiento	YEAR
portada_ruta	VARCHAR(255)

notas

Campo	Tipo
id_nota	INT (PK)
contenido	TEXT
fecha	TIMESTAMP

canciones

Campo	Tipo
id_cancion	INT (PK)
titulo	VARCHAR(150)
duracion_segundos	INT
ruta_archivo	VARCHAR(255)

playlists

Campo	Tipo
id_playlist	INT (PK)
nombre	VARCHAR(100)
descripcion	TEXT
created_at	TIMESTAMP

1. **TIMESTAMP**: es un tipo de dato que registra el instante exacto (fecha y hora) en que ocurre un evento, como la creación o modificación de un registro
2. **PK**: Llave primaria (Primary Key)

Relaciones

rel_tracks_album

Campo	Tipo
id_album	INT (PK)
id_cancion	INT (PK)
numero_pista	INT

rel_art_track

Campo	Tipo
id_artista	INT (PK)
id_cancion	INT (PK)
rol	VARCHAR(50)

rel_tracks_playlist

Campo	Tipo
id_playlist	INT (PK)
id_cancion	INT (PK)
posicion	INT
agregado_en	TIMESTAMP

rel_art_album

Campo	Tipo
id_artista	INT (PK)
id_album	INT (PK)

rel_user_playlist

Campo	Tipo
id_usuario	INT (PK)
id_playlist	INT (PK)
es_propietario	BOOLEAN

rel_notes_album

Campo	Tipo
id_nota	INT (PK)
id_album	INT (PK)

1. BOOLEAN: es un tipo de dato que solo puede almacenar dos valores lógicos: verdadero (TRUE) o falso (FALSE)

Explicación de las Relaciones

- `rel_tracks_album`: relaciona el `id_cancion` con el `id_album`, indicando además el número de pista (`numero_pista`) de la canción dentro del álbum.
- `rel_tracks_playlist`: relaciona el `id_cancion` con el `id_playlist`, guardando su posición en la lista y cuándo se agregó (`agregado_en`).
- `rel_art_track`: relaciona el `id_artista` con el `id_cancion`, y el campo `rol` indica su participación (por ejemplo, intérprete o compositor).
- `rel_art_album`: relaciona el `id_artista` con el `id_album`, es decir, qué artistas pertenecen a cada álbum.
- `rel_notes_album`: relaciona el `id_nota` con el `id_album`, permitiendo asociar notas a un álbum.
- `rel_user_playlist`: relaciona el `id_usuario` con el `id_playlist`, y `es_propietario` indica si el usuario es dueño de la playlist.

Ejemplo: ¿cómo se relacionan los registros?

artistas	
id_artista	nombre
1	Daft Punk

albumes		
id_album	titulo	anio
10	Discovery	2001

canciones		
id_cancion	titulo	duracion
100	One More Time	320
101	Digital Love	298

⇓ las tablas de relación los conectan con el álbum ⇓

rel_art_album	
id_artista	id_album
1	10

Daft Punk → Discovery

rel_tracks_album		
id_album	id_cancion	numero_pista
10	100	1
10	101	2

One More Time y Digital Love → Discovery

El álbum completo: uniendo tablas

¿Está incompleta la información del álbum?

No: así **optimizamos espacio**. Cada dato se guarda una sola vez en su tabla, y al consultarlo **unimos columnas de distintas tablas** para reconstruir el registro completo.

Resultado final (después de la consulta SQL)

album	artista	anio	pista	cancion
Discovery	Daft Punk	2001	1	One More Time
Discovery	Daft Punk	2001	2	Digital Love

Consulta SQL

```
1      SELECT al.titulo, ar.nombre,  
2             al.anio_lanzamiento,  
3             rt.numero_pista, c.titulo  
4  FROM  albums al  
5  JOIN  rel_art_album ra  
6        ON ra.id_album = al.id_album  
7  JOIN  artistas ar  
8        ON ar.id_artista = ra.id_artista  
9  JOIN  rel_tracks_album rt  
10       ON rt.id_album = al.id_album  
11  JOIN  canciones c  
12       ON c.id_cancion = rt.id_cancion
```

En lenguaje natural

- Toma las columnas titulo, nombre, anio_lanzamiento, numero_pista y titulo de las tablas albums, artistas, rel_tracks_album y canciones.
- Únelas usando las tablas de relación (rel_art_album y rel_tracks_album).
- Donde el id_album sea igual a 10.

Integración con Python

Código en Python

Funciones explicadas

Archivos

El proyecto se dividio en 3 archivos **Python**:

1. funciones.py

Las funciones que constantemente usaremos en el programa

2. conexion.py

La conexión con la base de datos.

3. main.py

El codigo donde integramos todo.

Librería `mysql.connector`

¿Qué es?

Librería oficial de MySQL para que Python pueda conectarse a la base de datos, ejecutar consultas SQL y recibir los resultados.

Funciones que usamos:

Función	Explicación
<code>mysql.connector.connect()</code>	Abre la conexión con la base de datos. La usamos dentro de <code>connect_to_database()</code> (archivo <code>conexion.py</code>).
<code>cnx.cursor()</code>	Crea el cursor, el objeto que envía las instrucciones SQL.
<code>cursor.execute(sql, valores)</code>	Ejecuta una consulta. Los <code>%s</code> se sustituyen por los valores, lo que evita inyección SQL.
<code>cursor.fetchone()</code>	Devuelve una sola fila (en <code>verify_user</code>).
<code>cursor.fetchall()</code>	Devuelve todas las filas (en las funciones <code>show_*</code>).
<code>cursor.lastrowid</code>	Entrega el ID del último registro insertado. (esto es un atributo, no una función)
<code>cnx.commit()</code>	Guarda los cambios de INSERT de forma permanente.

Integración con Python

Código en Python

Funciones explicadas

Archivos del Proyecto

El proyecto se dividió en 3 archivos principales de **Python**:

1. `conexion.py`
Establece y gestiona la conexión con la base de datos MySQL.
2. `funciones.py`
Contiene las funciones modulares para operaciones CRUD y lógica del negocio.
3. `main.py`
Integración del flujo principal y menú de usuario.

Consultas y Reportes de Datos

`show_artists()`, `show_tracks()`, `show_playlists()`

- **Propósito:** Recuperan registros formateados usando `cursor.fetchall()`.
- **Formato especial:** Convierte la duración de las canciones a minutos (`duracion // 60`).

`show_albums()`

- **Propósito:** Muestra una vista jerárquica del catálogo agrupando canciones.
- **SQL Avanzado:** Emplea `LEFT JOIN` entre `albumes`, `rel_tracks_album` y `canciones`.

Código Fuente: conexion.py

```
1  import mysql.connector
2  from mysql.connector import Error
3
4  def connect_to_database():
5      try:
6          cnx = mysql.connector.connect(
7              host="localhost",
8              user="root",
9              password="joshwbe",
10             database="music_manager",
11             port=3306
12         )
13         if cnx.is_connected():
14             print("Conexion exitosa a la base de datos music_manager!")
15             return cnx
16         except Error as error:
17             print(f"Error al conectar con MySQL: {error}")
18             return None
19
```

Gestión de Usuarios

`add_user(username, password_hash)`

- **Propósito:** Registra un nuevo usuario en la base de datos.
- **Funcionamiento:** Ejecuta la instrucción `INSERT INTO usuarios` y confirma los cambios con `cnx.commit()`.

`verify_user(username, password_hash)`

- **Propósito:** Autentica las credenciales de un usuario.
- **Funcionamiento:** Utiliza `SELECT id_usuario`. Si existe coincidencia, retorna el ID del usuario con `cursor.fetchone()`; de lo contrario, devuelve `None`.

Artistas, Álbumes y Canciones

`add_artist(nombre)`

- **Propósito:** Inserta un nuevo registro en la tabla artistas.

`add_album(titulo, descripcion, anio_lanzamiento, portada_ruta)`

- **Propósito:** Registra un álbum y utiliza `cursor.lastrowid` para obtener y retornar el ID generado.

`add_track(titulo, duracion_segundos, ruta_archivo)`

- **Propósito:** Inserta una pista en la tabla canciones y devuelve su ID único (`id_cancion`).

Relaciones y Playlists

`link_track_to_album(id_album, id_cancion, numero_pista)`

- **Propósito:** Asocia una canción a un álbum guardando su número de pista en la tabla intermedia `rel_tracks_album`.

`add_playlist(nombre, id_usuario, descripcion)`

- **Propósito:** Crea una lista de reproducción e inserta la relación de propiedad en `rel_user_playlist`.
- **Transacción segura:** Aplica `cnx.rollback()` en el bloque `except` para desestimar cambios si alguna consulta falla.

Consultas y Reportes de Datos

`show_artists()`, `show_tracks()`, `show_playlists()`

- **Propósito:** Recuperan registros formateados usando `cursor.fetchall()`.
- **Formato especial:** Convierte la duración de las canciones a minutos (`duracion // 60`).

`show_albums()`

- **Propósito:** Muestra una vista jerárquica del catálogo agrupando canciones.
- **SQL Avanzado:** Emplea `LEFT JOIN` entre `albumes`, `rel_tracks_album` y `canciones`.

Patrón de Diseño de las Funciones

Todas las funciones desarrolladas en `funciones.py` siguen un patrón de arquitectura constante:

1. **Obtención de conexión:** Invocación a `connect_to_database()`.
2. **Manejo de excepciones:** Estructura `try-except-finally`.
3. **Seguridad SQL:** Consultas con marcadores `%s` para evitar inyección SQL.
4. **Liberación de recursos:** Cierre explícito mediante `cursor.close()` y `cnx.close()` en `finally`.

Estructura del Archivo `main.py`

Propósito General

Actúa como el punto de entrada de la aplicación, gestionando la interfaz de consola, la autenticación y la navegación mediante menús interactivos.

Funciones de Utilidad:

- `limpiar_pantalla()`: Ejecuta `cls` (Windows) o `clear` (Unix) usando el módulo `os`.
- `mostrar_encabezado()`: Imprime la cabecera visual del sistema.

Integración de Módulos:

- Importa todas las funciones de operaciones desde `funciones.py`.
- Utiliza `time.sleep()` para mejorar la experiencia de usuario.

Flujo de Control: Autenticación y Navegación

`main()` - Pantalla de Acceso

- Controla el inicio de sesión con `verify_user()` y el registro con `add_user()`.
- Si las credenciales son válidas, concede acceso enviando el ID del usuario a `menu_principal()`.

`menu_principal()` - Gestión Interactiva

- Ofrece un menú numérico (opciones 0 a 9) para ejecutar las operaciones del sistema.
- Sanitiza las entradas del usuario (ej. conversión de texto a `int` para la duración de canciones e IDs).
- Maneja excepciones con `try-except ValueError` para evitar caídas del programa ante datos incorrectos.

Código Fuente: Inicialización (main.py)

```
1  import os
2  import time
3  from funciones import (add_artist, add_user, add_album, add_track,
4  add_playlist, link_track_to_album, verify_user,
5  show_albums, show_artists, show_tracks, show_playlists)
6
7  def limpiar_pantalla():
8  os.system('cls' if os.name == 'nt' else 'clear')
9
10 if __name__ == '__main__':
11     # Punto de entrada de la aplicacion
12     main()
13
```